

Backlink Admissibility in Cyclic Proofs for the Calculus of Inductive Constructions

Gavin Mendel-Gleason

Scidonia Limited
gavin@scidonia.ai

Abstract. Cyclic proofs close goals by backlinking to previously-seen configurations, discovering induction principles post-hoc rather than prescribing them upfront. We prove that backlinks are *admissible* in the Calculus of Inductive Constructions (CIC): every valid cyclic proof can be read back to a standard CIC term that uses only the native fixpoint binder $()$ for recursion and is observationally indistinguishable from the root goal. The proof is a direct corollary of a contextual-equivalence theorem mechanised in Rocq with 0 axioms.

1 Introduction

Cyclic proof theory [1,2] extends standard proof systems with backlink rules that allow a premise to reference a previous goal, forming a directed cycle. The cycle is sound if it satisfies a global *trace condition*: every cycle must contain a *progress event*—a case-split on a structurally smaller term. Brotherston and Simpson prove that for first-order logic with inductive definitions, backlinks are *admissible*: every cyclic proof corresponds to a standard proof with an explicit induction rule.

This paper extends their result to the Calculus of Inductive Constructions (CIC) [3,4], the type theory underlying Rocq. We prove that every rooted, ranked cyclic proof can be residualised to a standard CIC term using only λ for recursion, and that the residual is observationally equivalent to the root goal.

2 Term calculus and reduction

Terms are given by the grammar:

$$t, u, A, B ::= x \mid i \mid \Pi x:A. B \mid \lambda x:A. t \mid t \ u \mid t \mid It \mid Ict \mid It\kappa b$$

where x ranges over de Bruijn indices, i over universe levels, I identifies an inductive family, c a constructor index, κ a dependent motive (a binder over the scrutinee), and $\mathbf{t}, \mathbf{b}$ are lists of terms.

We write $\rightarrow^*$ for the reflexive-transitive closure of $\rightarrow$, and $t \Downarrow v$ (“ t terminates to v ”) for $t \rightarrow^* v$ with v a value (an abstraction, a constructor, or a neutral term). Multi-argument application $\mathbf{apps} \ t \ [u_1, \dots, u_n]$ is defined inductively: $\mathbf{apps} \ t \ [] = t$ and $\mathbf{apps} \ t \ (u :: \mathbf{u}) = \mathbf{apps} \ (t \ u) \ \mathbf{u}$.

Operational semantics. We use a call-by-name (CBN) reduction relation $t \rightarrow t'$ defined by:

- **β -reduction:** $(\lambda x:A. t) u \rightarrow t[u/x]$.
- **Fixpoint unfolding:** $t \rightarrow t[t/0]$. (The binder 0 in t refers to the recursive self-reference.)
- **ι -reduction (case-of-constructor):** $I(Ica)\kappa b \rightarrow \text{apps}(b[c]) a$ where $b[c]$ selects the c -th branch.
- **Congruence:** Reduction applies under applications and case scrutinees: if $t \rightarrow t'$ then $t u \rightarrow t' u$ and $It\kappa b \rightarrow It'\kappa b$.

A term t *terminates to* a value v , written $t \Downarrow v$, if $t \rightarrow^* v$ and v is in weak-head normal form (an abstraction, a constructor, or a neutral term).

3 Cyclic sequent calculus

Configurations are typing judgements $\Gamma \vdash t : A$. A *configuration graph* b is a finite map from vertices $v \in \{0, \dots, n-1\}$ to configurations $\ell(b, v)$ and successor lists $\sigma(b, v)$. A graph is a *rooted pre-proof* with root v_0 if every vertex satisfies a local sequent rule drawn from the following:

Asynchronous rules (deterministic). Applied eagerly. Each reduces the goal to a single premise by one computation step:

$$\frac{\Gamma \vdash t[u/x] : A}{\Gamma \vdash (\lambda x:A. t) u : A} \beta \qquad \frac{\Gamma \vdash t[t/0] : A}{\Gamma \vdash t : A} \text{FIX}$$

$$\frac{\Gamma \vdash \text{apps}(b[c]) a : A}{\Gamma \vdash I(Ica)\kappa b : A} \iota$$

Synchronous rule (choice point). A case-split on a neutral scrutinee x creates one premise per constructor c of I , each in an extended context with fresh constructor arguments $\mathbf{y}_c$:

$$\frac{\Gamma, \mathbf{y}_0 \vdash t_0 : A \quad \dots \quad \Gamma, \mathbf{y}_n \vdash t_n : A}{\Gamma \vdash Ix\kappa b : A} \text{SPLIT}$$

Cyclic backlink (closure). $\frac{\Delta \vdash \sigma :_s \Gamma}{\Delta \vdash t : A} \text{BACK}$ where t is an instance of a previously-seen configuration $\Gamma \vdash t_0 : A$ under the explicit substitution σ .

Global progress condition. A rooted pre-proof is a *rooted cyclic proof* if every directed cycle contains at least one SPLIT edge. This is witnessed by a budget-trace ranking: vertices are lifted to pairs (v, k) where $k \in [0, |V|]$. SPLIT edges consume budget ($k \mapsto k-1$); all other edges preserve it. The lifted graph must be acyclic. We write $\mathcal{C}(b, v_0)$ for this property.

4 Residualisation

The backlink rule appears only in the proof graph. A *residualisation* function $\mathcal{R}(b, v)$ reads back the graph into a CIC term by traversing the successors of v and replacing backlinks with `binders`:

- A vertex with no successors returns its own term.
- A vertex with one successor applies the driving step (unfold, β -reduce, apply constructor) to the successor’s residual.
- A vertex that is the target of a backlink becomes a `where` whose body is the residual of its successors, with recursive calls referencing the `where` binder.

The result is a well-typed CIC term using only standard constructors (λ , application, `,`, `,`). No backlink constructs remain.

5 CIU equivalence

The soundness guarantee is *CIU equivalence* (contextual equivalence under all closing substitutions):

$$t \approx_{\text{ciu}} u \equiv \forall \sigma, v. (t[\sigma] \Downarrow v \iff u[\sigma] \Downarrow v)$$

The mechanised CIU theorem (companion paper, under review) establishes that residualisation preserves equivalence:

Theorem 1 (CIU soundness). *If b is a rooted, ranked cyclic proof with root v and $\ell(b, v) = \Gamma \vdash t : A$, then*

$$t \approx_{\text{ciu}} \mathcal{R}(b, v).$$

The proof (mechanised in Rocq, 0 axioms) is by fuel induction on the residualiser. Congruence lemmas lift CIU through all nine term constructors. The trace condition excludes non-terminating self-loops.

6 Admissibility

Theorem 2 (Backlink admissibility). *For every configuration graph b and vertex v ,*

$$\mathcal{C}(b, v) \implies \exists t_{\text{std}}, t \approx_{\text{ciu}} t_{\text{std}} \wedge t_{\text{std}} = \mathcal{R}(b, v)$$

where $\ell(b, v) = \Gamma \vdash t : A$.

In words: every rooted cyclic proof can be read back to a standard CIC term (using only `for` for recursion) that is observationally indistinguishable from the root goal.

Proof. Take $t_{\text{std}} = \mathcal{R}(b, v)$. By construction, t_{std} uses only standard CIC constructors (Section 4). The equality $t \approx_{\text{ciu}} t_{\text{std}}$ follows from Theorem 1. $\square$

The mechanised proof (`BacklinkAdmissible.v`) is a four-line corollary of the CIU theorem.

7 Discussion

The admissibility transformation is the same residualisation that the proof graph already provides. The CIU theorem proves equivalence; the residualiser eliminates backlinks; the result follows trivially.

Compared to Brotherston and Simpson’s result for first-order logic, our proof is simpler because (a) the graph is finite, and (b) residualisation is a syntactic transformation that explicitly replaces backlinks with `←`, rather than requiring a semantic argument about proof unfoldings.

Acknowledgements. Thanks to James Brotherston for the question that motivated this paper.

References

1. Brotherston, J.: Cyclic proofs for first-order logic with inductive definitions. In: TABLEAUX (2006)
2. Brotherston, J., Simpson, A.: Sequent calculi for induction and infinite descent. In: LICS (2011)
3. Coquand, T., Huet, G.: The calculus of constructions. *Information and Computation* **76**(2–3), 95–120 (1988)
4. Paulin-Mohring, C.: Inductive definitions in the system Coq: Rules and properties. In: *Typed Lambda Calculi and Applications*. pp. 328–345. Springer (1993)